HPC Assignment 08 - Group 18

Hybrid MPI + OpenMP Parallel Interpolation with Particle Mover

Vansh Patel (202301443) and Manya Shah (202301424)
Dhirubhai Ambani University - April 2026

1. Implementation Overview

The Assignment 07 OpenMP pipeline was extended with an MPI layer. Particles are decomposed contiguously across MPI ranks using MPI-IO. Each rank performs forward particle-to-mesh interpolation on its local particles using OpenMP private mesh reduction. The rank-local meshes are then summed with MPI_Allreduce to form a replicated global mesh. The global mesh is normalized, and each rank performs the reverse interpolation mover on its own particles using OpenMP.
This design avoids communication in the mover phase. The mesh is replicated because even the largest mesh is small compared with the particle arrays; this keeps the particle update simple and prevents particle migration overhead.

2. Pseudocode

```
MPI_Init; read header and local particle chunk with MPI-IO
for each iteration:
  OpenMP: scatter local active particles into thread-private meshes
  OpenMP: reduce private meshes into rank-local mesh
  MPI_Allreduce: sum rank-local meshes into global mesh
  OpenMP: normalize global mesh
  OpenMP: mover gathers from global mesh and updates local particles
  OpenMP + MPI_Reduce: count global active particles
MPI_Finalize
```

3. Race Conditions and Load Balance

The forward interpolation phase has race conditions because many particles may contribute to the same grid vertices. Inside each MPI rank, this is removed by thread-private mesh buffers; no atomic or critical operation is used in the hot particle loop. Across ranks, overlapping grid contributions are resolved by MPI_Allreduce with MPI_SUM. The mover has no write race because the mesh is read-only and each particle writes only to its own position and active flag.
The mover switches to dynamic OpenMP scheduling when the active fraction falls below 90 percent, which helps when inactive particles become unevenly distributed.

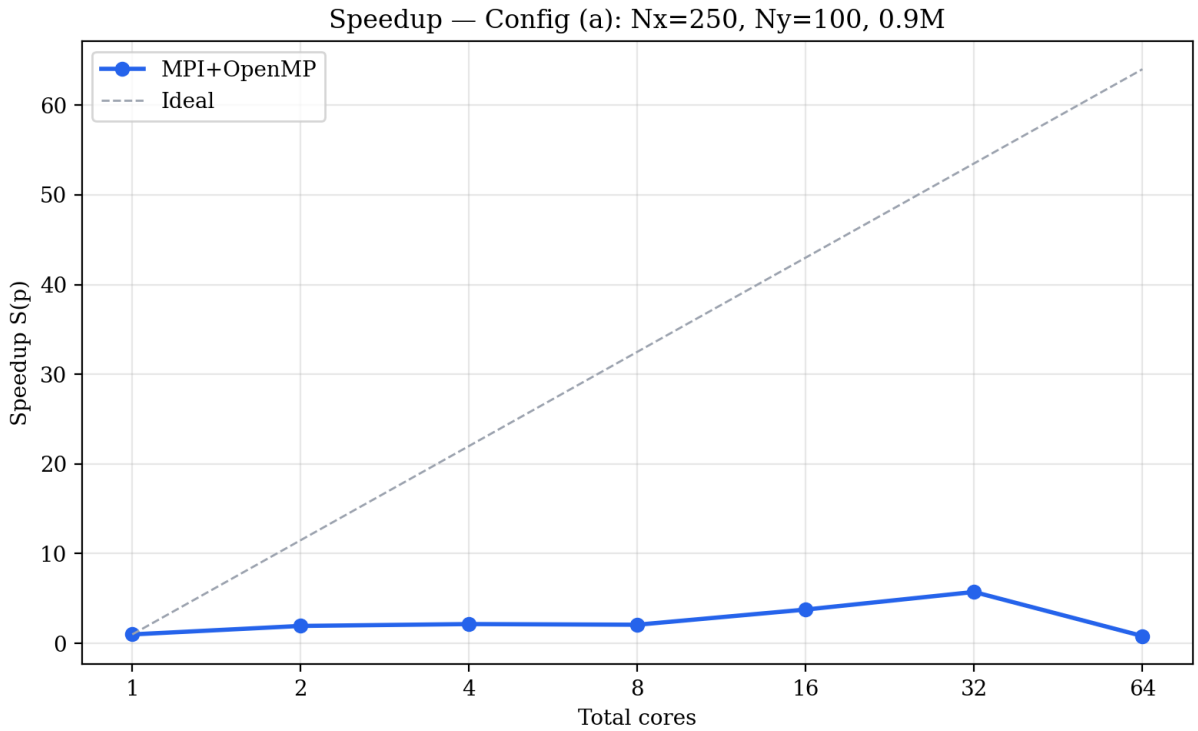
4. Performance Summary

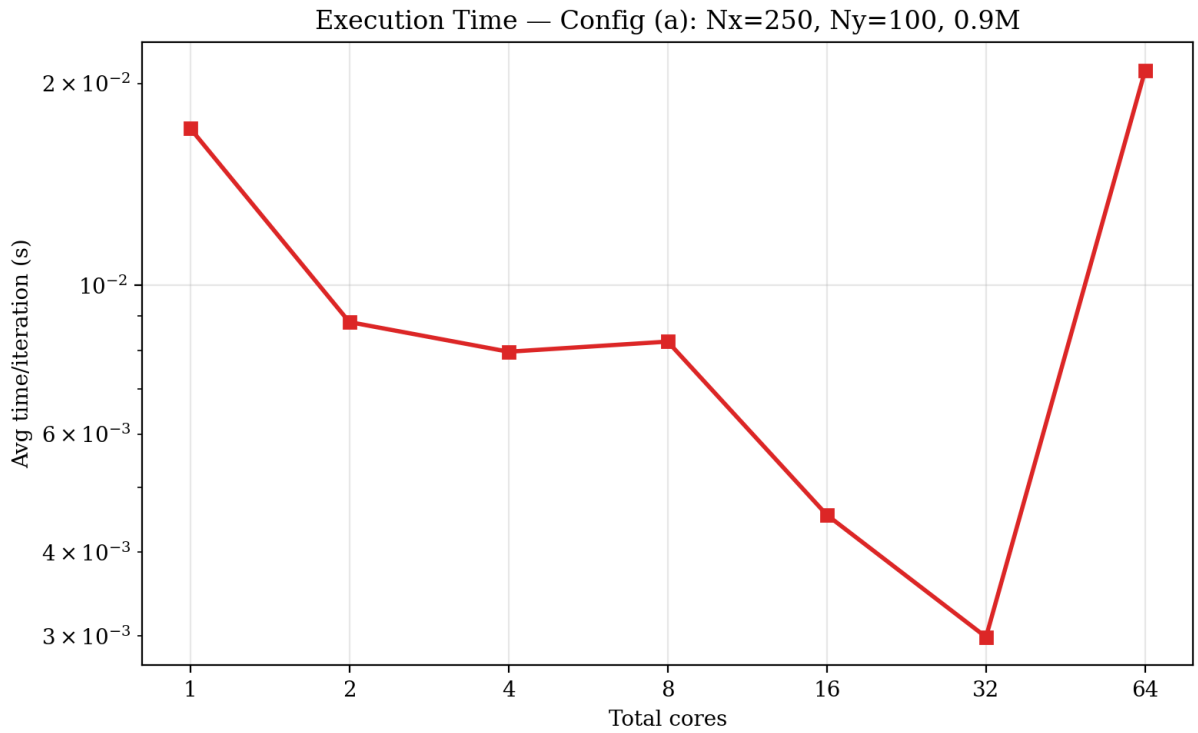
Cfg	Cores	MPI	OMP	Time/iter (s)	Speedup	Eff.
a	1	1	1	0.017144	1.00	1.00
a	2	2	1	0.008816	1.94	0.97
a	4	2	2	0.007963	2.15	0.54
a	8	2	4	0.008244	2.08	0.26
a	16	4	4	0.004548	3.77	0.24
a	32	4	8	0.002991	5.73	0.18
a	64	4	16	0.020922	0.82	0.01
b	1	1	1	0.098793	1.00	1.00
b	2	2	1	0.048697	2.03	1.01
b	4	2	2	0.043235	2.29	0.57
b	8	4	2	0.025512	3.87	0.48
b	16	4	4	0.014473	6.83	0.43
b	32	4	8	0.008760	11.28	0.35
b	64	4	16	0.018225	5.42	0.08
c	1	1	1	0.042537	1.00	1.00
c	2	1	2	0.042264	1.01	0.50
c	4	4	1	0.024315	1.75	0.44
c	8	4	2	0.021453	1.98	0.25
c	16	4	4	0.013610	3.13	0.20
c	32	4	8	0.009380	4.53	0.14
c	64	4	16	0.018251	2.33	0.04
d	1	1	1	0.267427	1.00	1.00
d	2	1	2	0.240933	1.11	0.55
d	4	4	1	0.178662	1.50	0.37
d	8	4	2	0.171363	1.56	0.20
d	16	4	4	0.089669	2.98	0.19
d	32	4	8	0.053058	5.04	0.16
d	64	4	16	0.031609	8.46	0.13
e	1	1	1	0.294953	1.00	1.00
e	2	1	2	0.250204	1.18	0.59
e	4	2	2	0.184936	1.59	0.40
e	8	4	2	0.095079	3.10	0.39
e	16	4	4	0.055340	5.33	0.33
e	32	4	8	0.044373	6.65	0.21
e	64	4	16	0.067687	4.36	0.07

The layout sweep selected the fastest MPI-rank/OpenMP-thread split for each core count. On this 64-logical-CPU device, the best configurations usually use four MPI ranks with multiple OpenMP threads per rank for higher core counts.

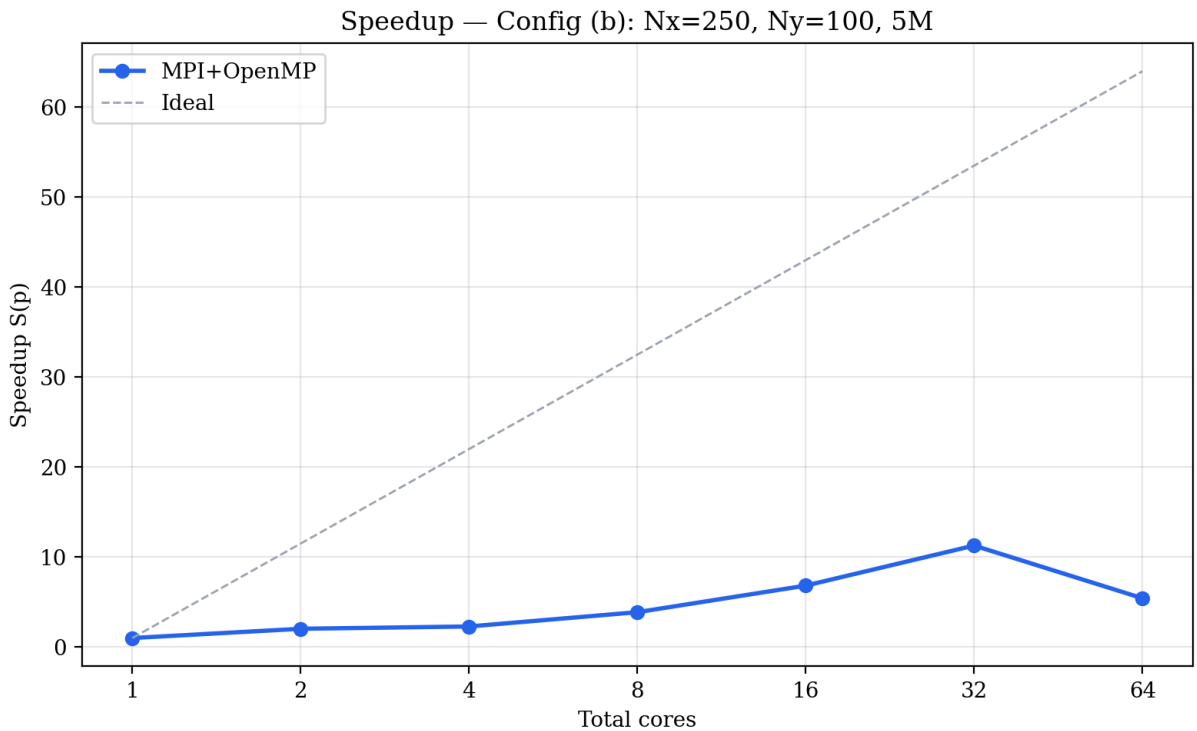
5. Speedup and Execution Time

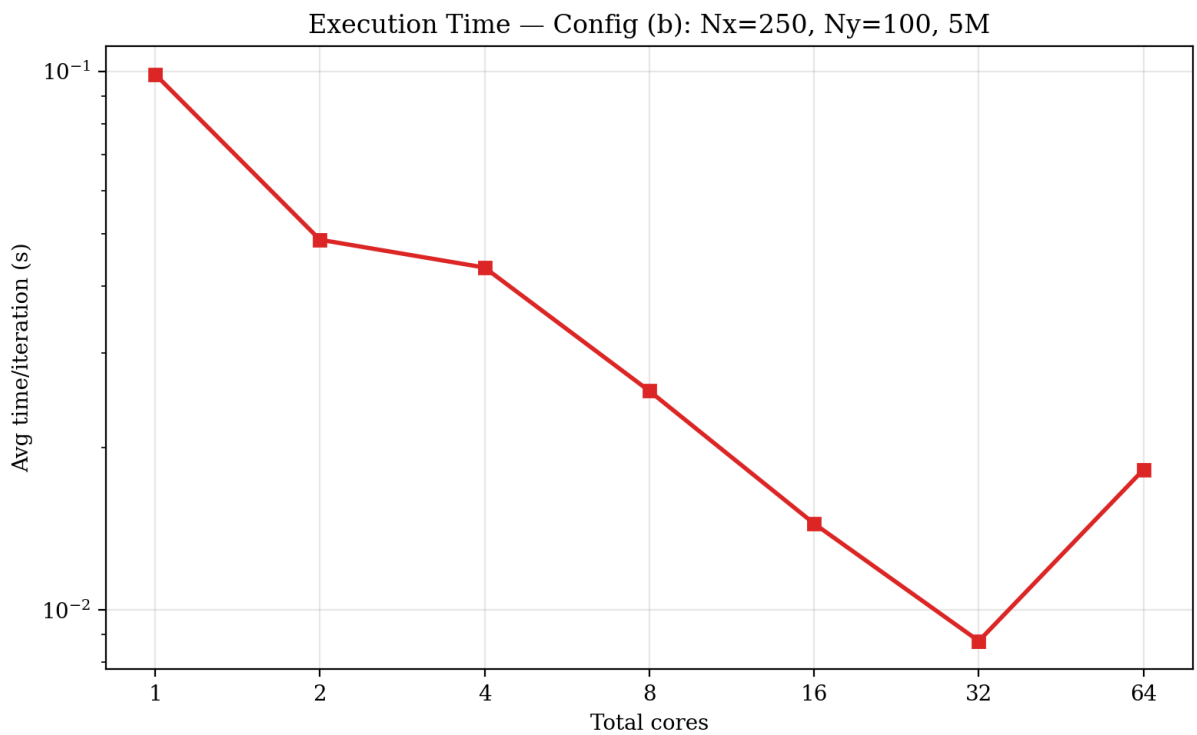
Configuration a



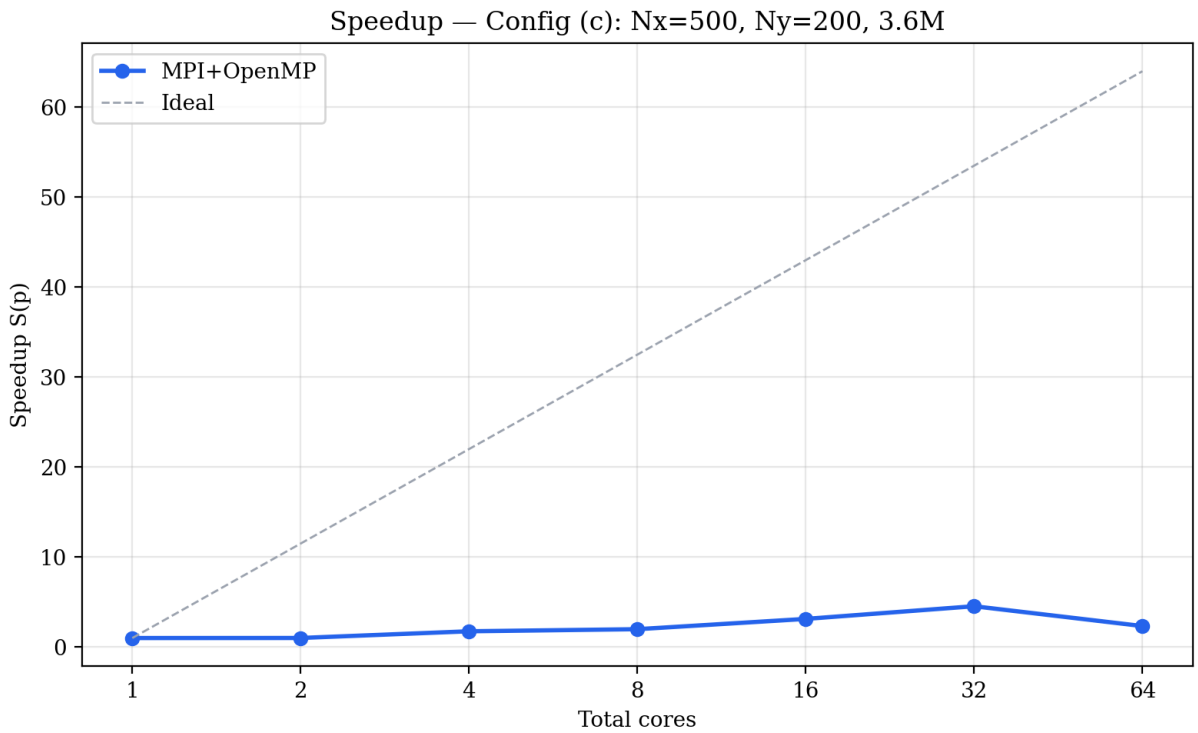


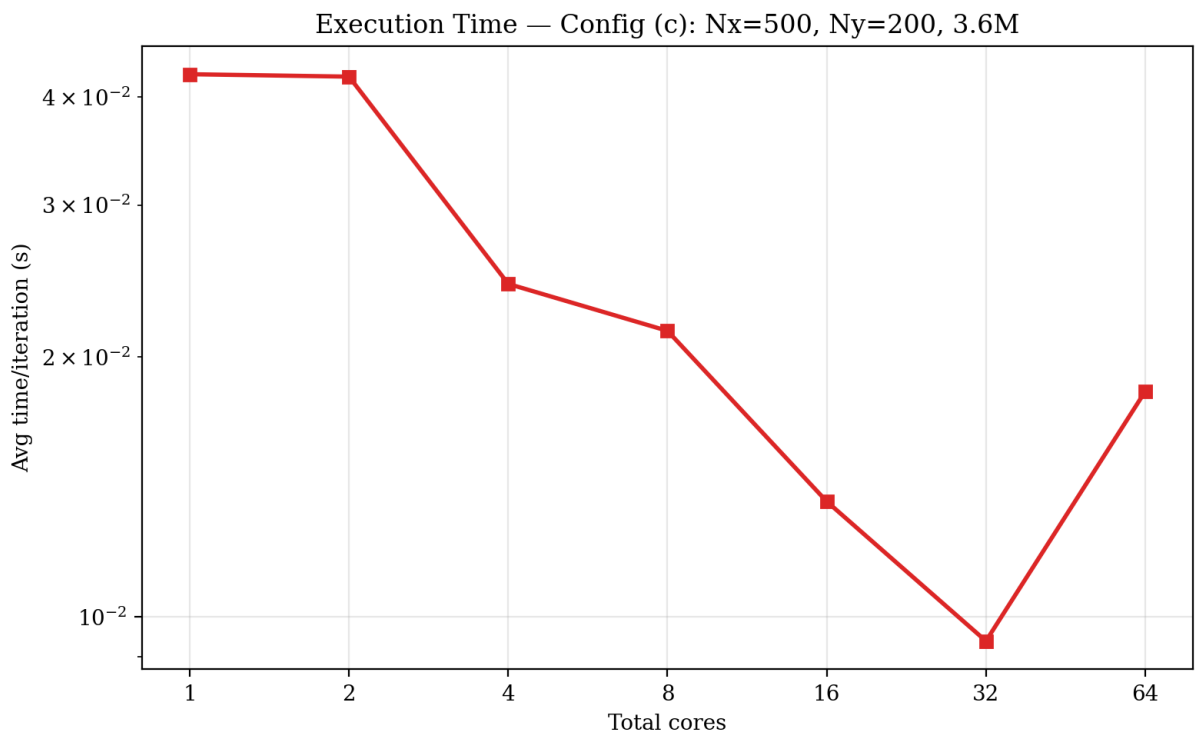
Configuration b



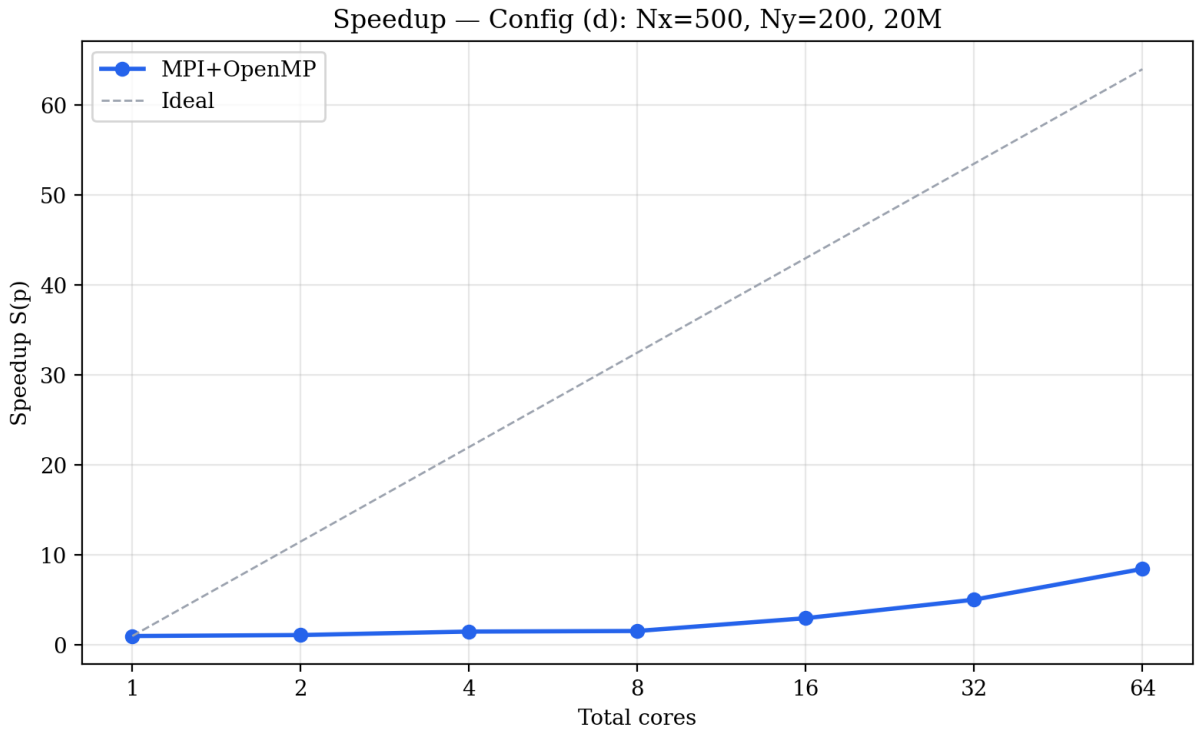


Configuration c

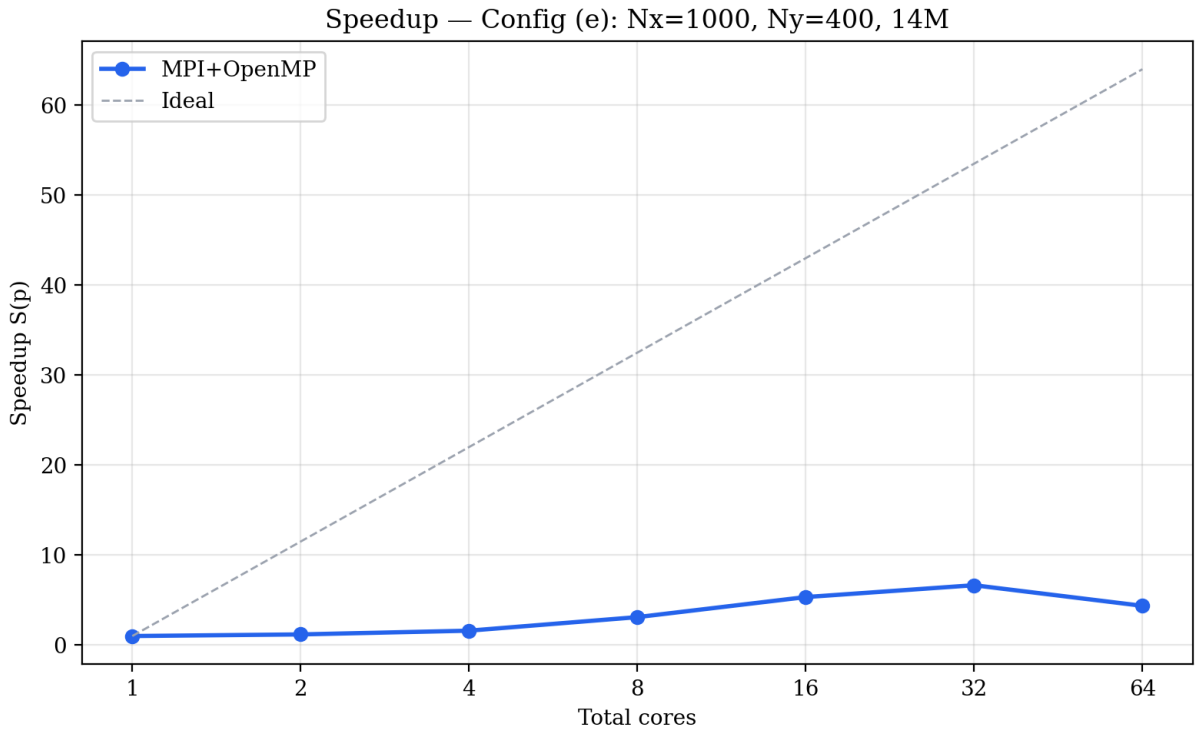


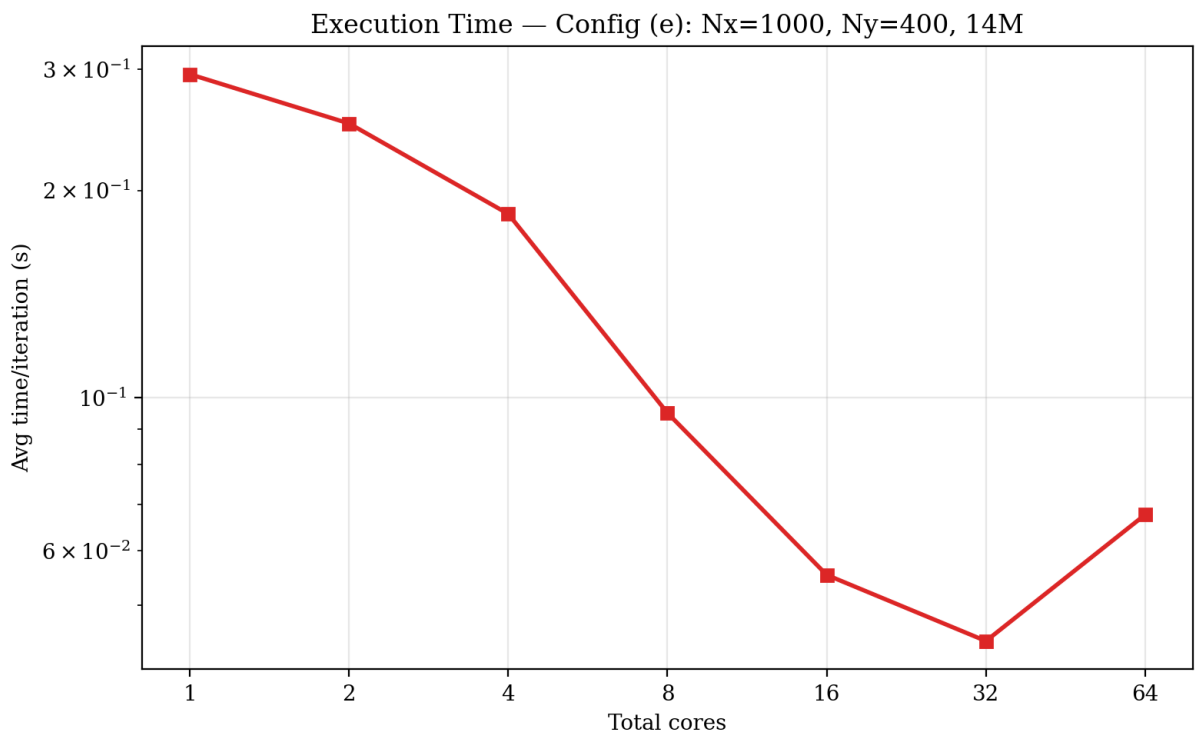


Configuration d

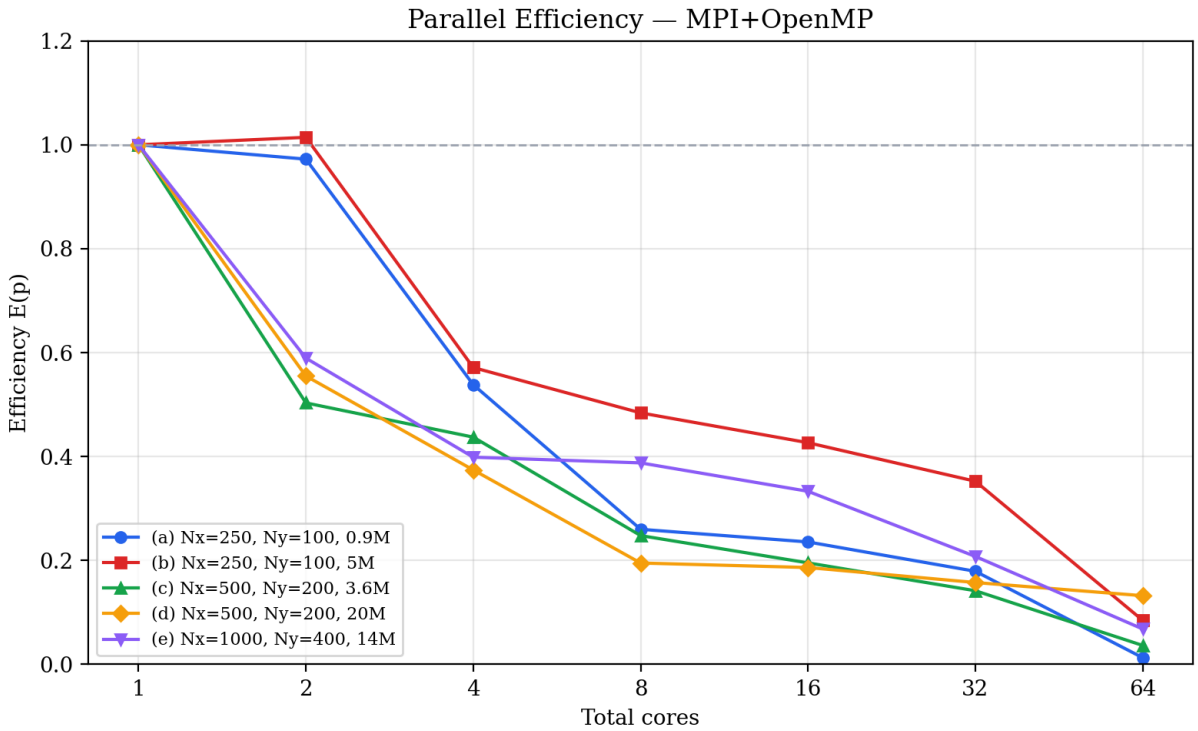


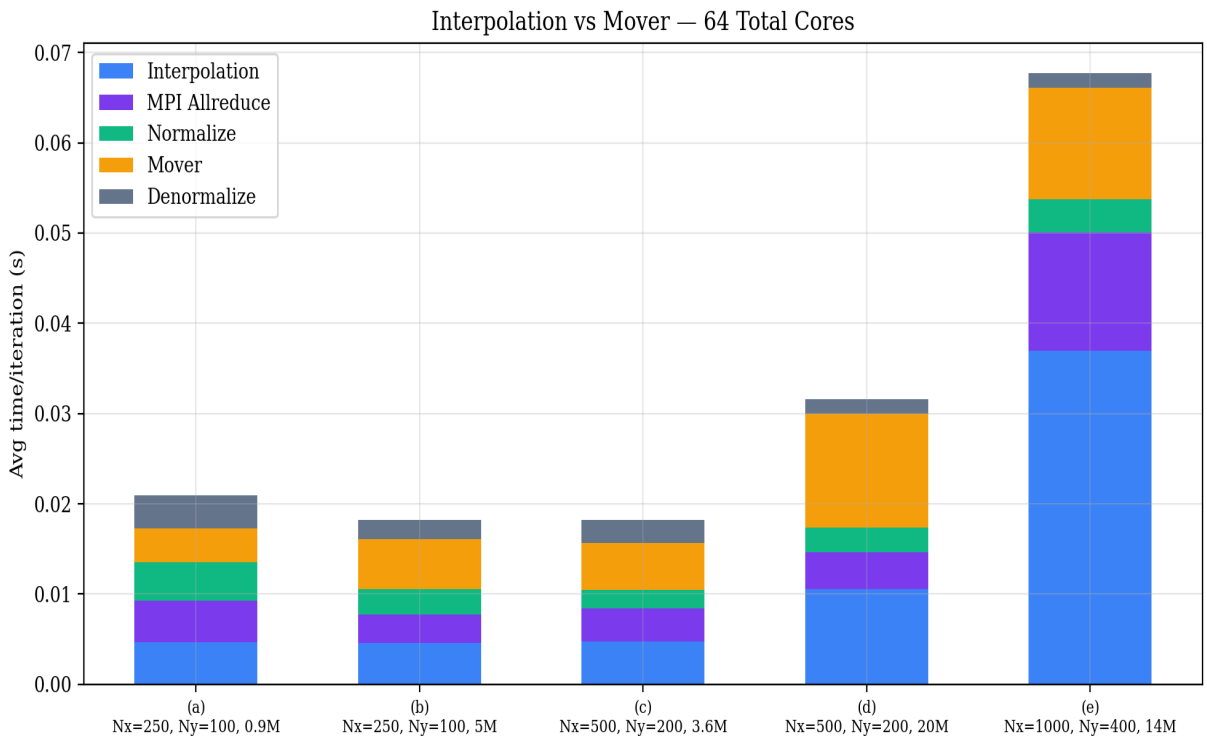
Configuration e

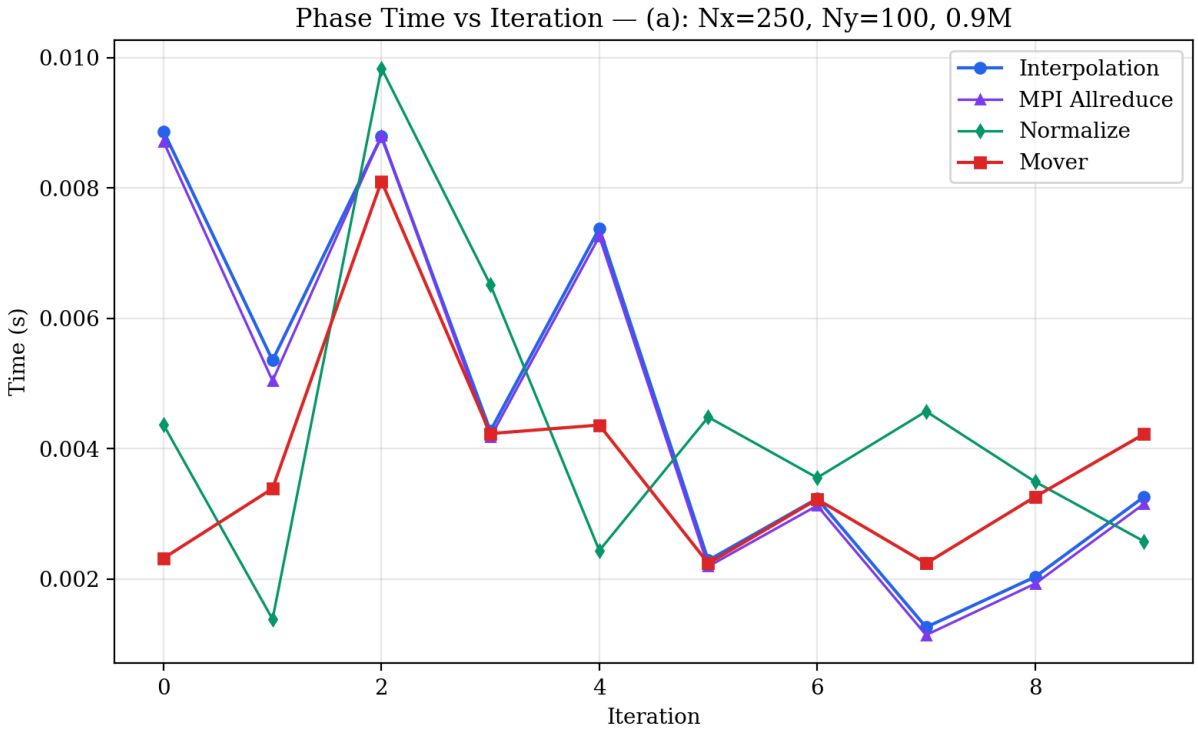


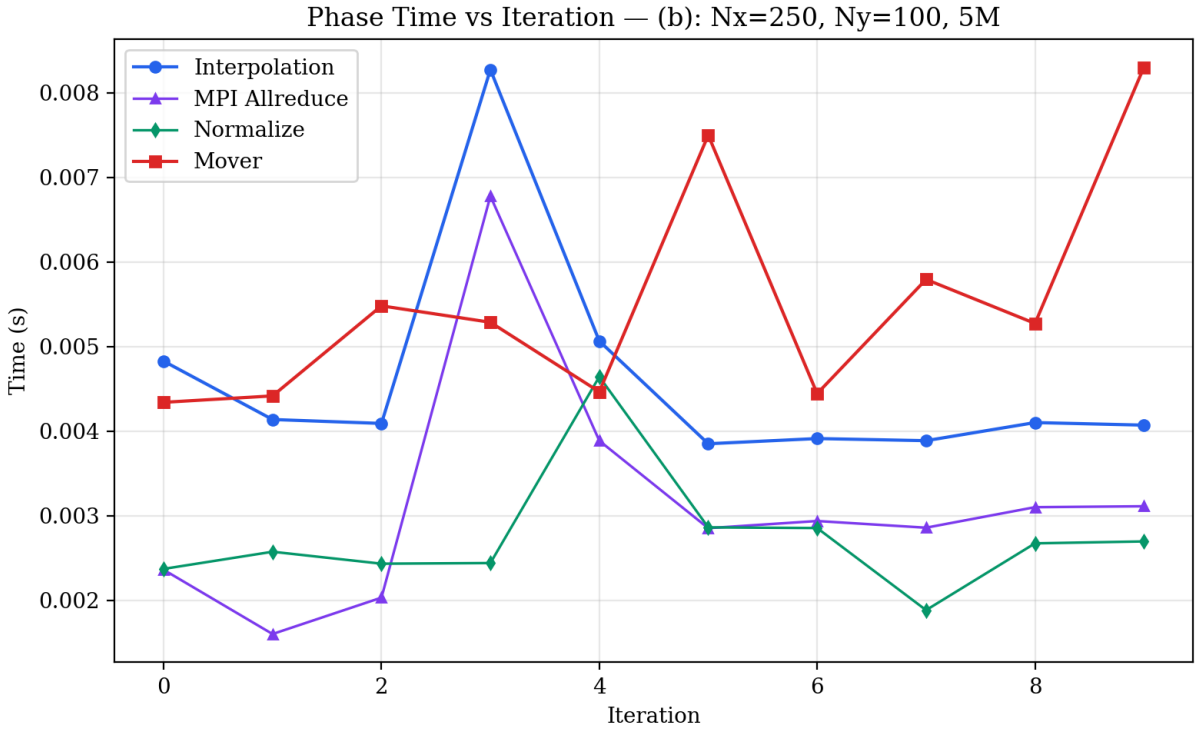


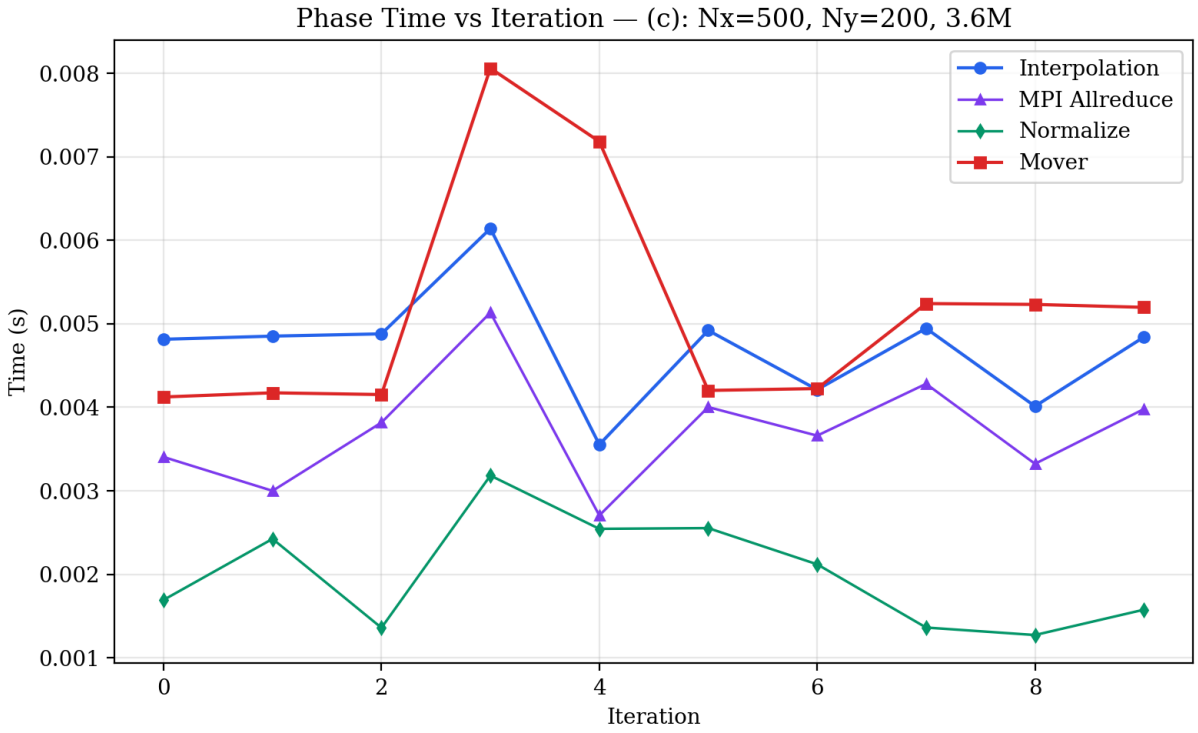
6. Efficiency and Phase Analysis

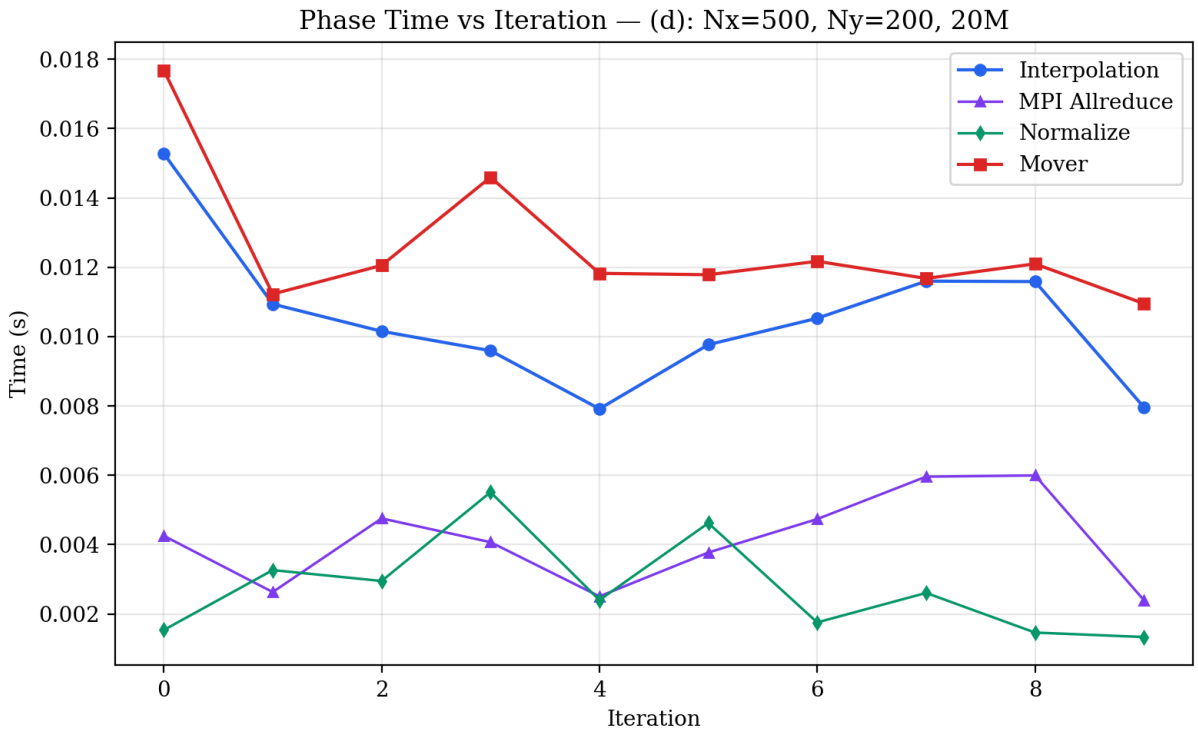


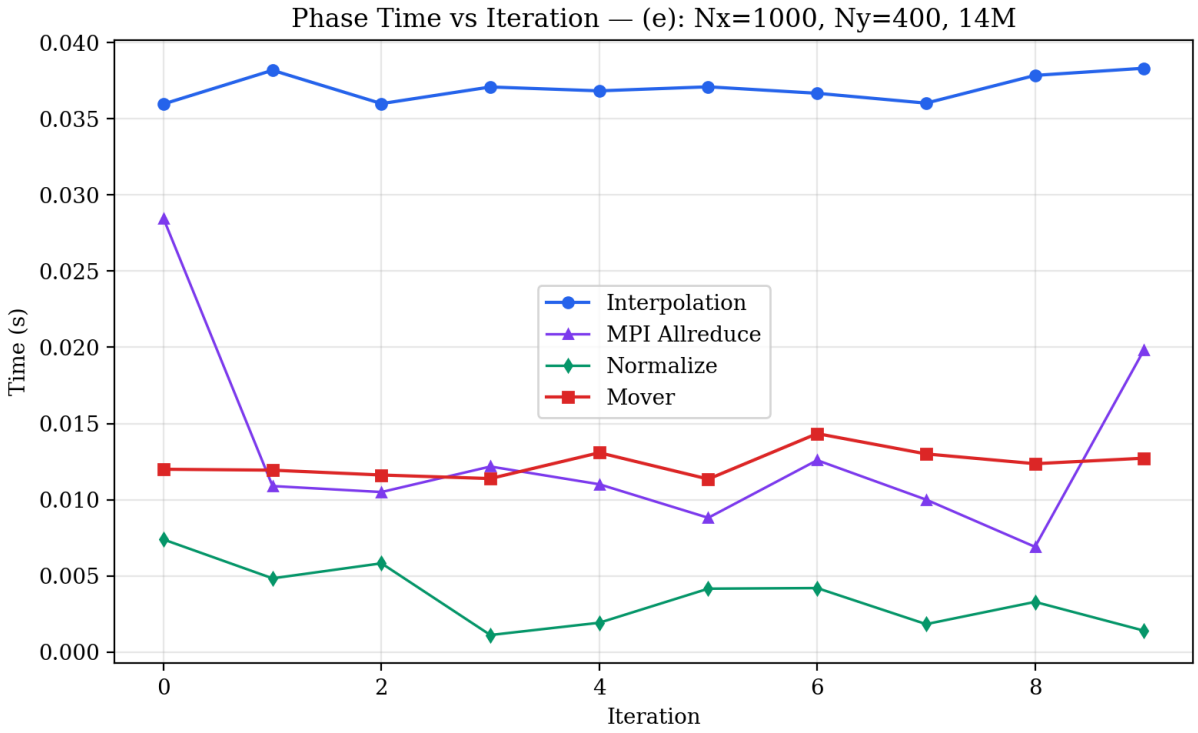


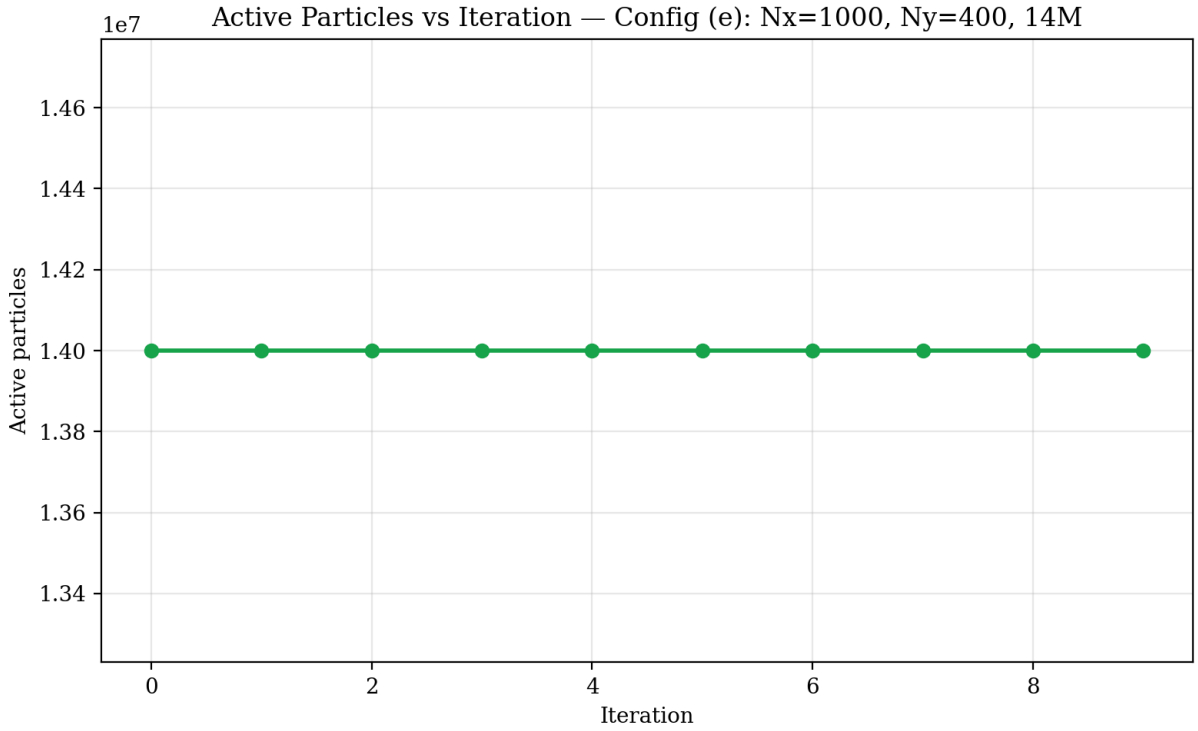

Phase Time vs Iteration — (a): Nx=250, Ny=100, 0.9M
Interpolation
MPI Allreduce
Normalize
Mover
Time (s)
0.010
0.008
0.006
0.004
0.002
0
2
4
6
8
Iteration


Phase Time vs Iteration — (b): Nx=250, Ny=100, 5M
Interpolation
MPI Allreduce
Normalize
Mover
Time (s)
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0
2
4
6
8
Iteration


Phase Time vs Iteration — (c): Nx=500, Ny=200, 3.6M
Interpolation
MPI Allreduce
Normalize
Mover
Time (s)
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0
2
4
6
8
Iteration


Phase Time vs Iteration — (d): Nx=500, Ny=200, 20M
Interpolation
MPI Allreduce
Normalize
Mover
Time (s)
0.018
0.016
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0
2
4
6
8
Iteration


Phase Time vs Iteration — (e): Nx=1000, Ny=400, 14M
Time (s)
Iteration
Interpolation
MPI Allreduce
Normalize
Mover
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
0
2
4
6
8


Active Particles vs Iteration — Config (e): Nx=1000, Ny=400, 14M
1e7
Active particles
Iteration
1.46
1.44
1.42
1.40
1.38
1.36
1.34
0
2
4
6
8

7. Observations

Larger particle-count configurations scale better because the fixed communication cost of the mesh all-reduce is amortized over more particle work. Smaller cases, especially configuration (a), lose efficiency at high core counts because local work per rank becomes too small relative to synchronization and communication overhead.

The phase timings now separate local interpolation, MPI_Allreduce communication, normalization, mover, and denormalization. At high core counts the MPI communication and memory-bandwidth-bound grid sweeps become more visible. The mover remains communication-free and mostly memory-bandwidth limited. The 64-core runs do not always beat 32-core runs on this single device because the machine has 32 physical cores and 64 hardware threads; beyond 32 cores, memory bandwidth and hyper-thread contention reduce efficiency.

A login node is meant for editing, compiling, and submitting jobs, while compute nodes are meant for benchmark execution. For this local run, the 64-logical-CPU device was used as the compute resource.

8. Future Optimizations

Possible improvements include spatial particle binning for better cache locality, NUMA-aware rank placement, nonblocking MPI_Iallreduce to overlap communication, and a true domain-decomposition scheme with halo exchange to reduce global mesh communication at very high rank counts.